Arquitectura de Redes

DRAFT

DRAFT

Copyright © 2019 Juan Marín Noguera, juan.marinn@um.es.

Esta obra es libre: puedes distribuirla y/o modificarla bajo los términos de la licencia Creative Commons Attribution-ShareAlike 3.0. Para ver una copia de esta licencia, visita https://creativecommons.org/licenses/by-sa/3.0/legalcode. Esta obra viene sin ninguna garantía, al máximo alcance permitido por la legislación aplicable.

Gracias por comprar este ejemplar. Al hacerlo estás ayudando a los refugiados de Atenas, pues los beneficios se destinan íntegramente a Project Elea, un grupo de voluntarios que trabajan con los residentes del campo de refugiados de Eleonas para mejorar los estándares de vida y el bienestar comunitario dentro de este. Ver https://projectelea.org/about-us/ para más información.

En el muy improbable caso de que el proyecto deje de recibir donaciones antes de enviar el dinero, este se destinaría a otra ONG.

Bibliografía:

- Arquitectura de Redes, Grado en Ingeniería Informática, Universidad de Murcia (Pedro M. Ruiz Martínez y Ramón J. Sánchez Iborra, 2019).
- RFC 1112 (https://www.ietf.org/standards/rfcs/).

Capítulo 1

Introducción

Una **red** es una colección de dispositivos autónomos, interconectados por medios físicos llamados **enlaces de comunicación**, y a través de los cuales se conectan siguiendo **protocolos**, que indican reglas y convenciones a seguir. Distinguimos:

- **Sistemas finales** o ***hosts***: Ordenadores, *tablets*, «*smart*»*phones*, etc. Se conectan a la red para acceder a servicios o datos disponibles en otros *hosts*.
- **Dispositivos de conmutación**: Conmutadores Ethernet o *switches*, encaminadores o *routers*, centralitas telefónicas, etc. Se conectan a varios enlaces o segmentos de red para facilitar el intercambio de datos y expandir el alcance de la red, retransmitiendo los datos que reciben por uno de sus puertos o interfaces a otro(s) puerto(s).

DRAFT

Retardos en el envío de mensajes:

De propagación Tiempo desde que se empieza a enviar un mensaje hasta que empieza a recibirse por el otro lado. Si el enlace tiene longitud ℓ y la velocidad de la luz en su interior es c, este es $P = \ell/c$.

De transmisión Tiempo que lleva transmitir un mensaje. Si el mensaje tiene tamaño p y la velocidad de transmisión es R, este es $T = p/R$.

De encolado Lo que tarda un dispositivo de comunicación desde que empieza a recibir un mensaje hasta que empieza a retransmitirlo, por tener que almacenarlo temporalmente.

Latencia Retardo total. Si un mensaje pasa por n dispositivos de comunicación con retardos de encolado Q_i y por $n+1$ enlaces con retardos respectivos de propagación y transmisión P_i y T_i, esto es $\sum_i (P_i + T_i) + \sum_i Q_i$.

1.1. Estrategias de conmutación

Conmutación de circuitos La red telefónica. Los dispositivos de red conectan puertos de entrada y salida físicamente. La comunicación tiene tres fases:

1. **Establecimiento**: El *host* que desea iniciar la comunicación indica a un dispositivo de conmutación al que está conectado con quién quiere comunicar, así como las propiedades de la comunicación, y este analiza la dirección para determinar por qué puerto seguir enviando esta indicación, si hay alguno libre que llegue al destino. Esto se repite hasta llegar el *host* final, que decide si acepta la conexión. Fase lenta.

2. **Comunicación**: Los *hosts* se comunican por el circuito, y los elementos intermedios se limitan a informar a estos en caso de desconexión brusca por algún error.

3. **Cierre**: El *host* que desea terminar la comunicación se lo indica al dispositivo de conmutación más cercano del circuito, y este mensaje se retransmite hasta que el circuito queda totalmente liberado.

El circuito es dedicado y con un ancho de banda fijo, y la transmisión es continua, pero si los equipos finales no hacen uso de la capacidad, esta se desperdicia, y si un enlace o dispositivo por el que pasa el circuito falla, la comunicación se interrumpe.

Conmutación de mensajes Los *routers* redirigen el mensaje por la interfaz adecuada usando información adicional en los mismos, como el identificador de destinatario, y la información de la que disponen, y usan *buffers* para almacenar los mensajes mientras la interfaz de salida está en uso (*store-and-forward*). La capacidad de los enlaces se usa de forma dinámica.

DRAFT

Conmutación de paquetes Como la de mensajes, pero los mensajes se dividen en trozos pequeños llamados paquetes, enrutados individualmente, para mejor rendimiento. En efecto, la latencia al enviar un mensaje de longitud m con información de enrutamiento de tamaño h, si no consideramos el tiempo de encolado, es

$$\sum_i (T_i + P_i) = \sum_i \left(\frac{m+h}{R_i} + P_i \right),$$

pero si lo dividimos en paquetes de longitud p, sea k el índice del segmento de red con menor tasa de transferencia ($R_k = \text{mín}\{R_i\}$), el retardo pasa a ser

$$\sum_i \left(\frac{p+h}{R_i} + P_i \right) + \frac{(m/p-1)(p+h)}{R_k},$$

donde el primer sumando es la latencia del primer paquete y el segundo es, una vez recibido el primer paquete, lo que tarda en llegar el resto. Si consideramos $h \ll p \ll m$, nos queda una latencia de

$$\frac{m}{R_k} + \sum_i P_i.$$

Con tiempo de encolado se obtiene un resultado análogo, y la conclusión es que lo ideal es usar un tamaño de paquete pequeño, pero no excesivamente pequeño. Esto además permite mayor tolerancia a fallos, pues si un enlace falla, el resto de paquetes pueden encaminarse por otros caminos. Por otro lado, la latencia de cada paquete puede variar según los retardos de encolado o el cambio de camino. A esta varianza se la llama ***jitter***.

Conmutación de circuitos virtuales Se emula la conmutación de circuitos con conmutación de paquetes. En la fase de establecimiento, se mandan paquetes con la información para el establecimiento de circuito, y se usa un **control de admisión** para garantizar un ancho de banda. En la fase de comunicación, los mensajes se dividen en paquetes, con un identificador de circuito virtual. Se intenta que los paquetes vayan siempre por el mismo camino, pero en caso de error de un dispositivo de conmutación o de un enlace, se puede cambiar a otro. Por ello, actualmente sólo se usan conmutación de paquetes y de circuitos virtuales.

1.2. Ganancia por multiplexión estadística

La **multiplexión** es el uso simultáneo de un canal de comunicaciones por varios usuarios, por ejemplo, usando distintas frecuencias o periodos de tiempo.

En Internet, las tasas de transferencia de los usuarios son muy variables, por lo que la conmutación de paquetes es muy efectiva como medio de multiplexión por intervalos de tiempo asignados dinámicamente. Esto es así porque los picos de transferencia no suelen coincidir, con lo que al ir agregando flujos a un canal, la tasa de envío por el canal se va estabilizando, y no es necesario proporcionar tanta velocidad como la suma del mayor pico de cada flujo.

Esto elimina las garantías de calidad de servicio: si el *buffer* de un dispositivo de comunicación es limitado y la velocidad de los flujos entrantes es mayor que la de salida, el *buffer* puede llenarse y hay pérdidas, y además, el retardo de encolado de un paquete puede variar, siendo proporcional al nivel de ocupación del *buffer*, aunque limitado por la relación entre su capacidad total y la velocidad de salida.

Dados N flujos con tasas de envío de $R_1(t), \ldots, R_N(t)$, se define la **ganancia por multiplexión estadística** (**SMG**, *Statistical Multiplexing Gain*) como

$$\mathrm{SMG} = \frac{\sum_i \max_t R_i(t)}{\max_t \sum_i R_i(t)} \geq 1.$$

Internet consigue grandes ganancias por multiplexación estadística, haciendo un uso muy eficiente de los enlaces, y esto, junto con la gran robustez y tolerancia a fallos, es por lo que usa conmutación de paquetes.

1.3. Arquitecturas de red

El software de una red de comunicaciones aborda problemas como codificación, sincronización, acceso a medio compartido, detección y corrección de errores, encaminamiento, garantía de extremo a extremo, etc., y para aliviar la complejidad de esto, las redes se organizan en **capas** o **niveles**, una encima de otra, de forma que una capa en una máquina habla siempre con la misma capa de otra y a través de la interfaz dada por la capa inferior.

Una **arquitectura de red** es un conjunto de protocolos y capas de define el funcionamiento de la red, con información suficiente para poder construir el hardware y software de capa capa pero sin indicar la manera concreta en que se comunica el software de distintas capas.

Las capas del modelo **OSI** son:

Física Se encarga exclusivamente de la transmisión de bits por un canal.

DRAFT

Enlace Se toma en medio de transmisión en bruto y se implementa lo necesario para evitar errores de transmisión: división de paquetes en tramas, códigos correctores o detectores de errores, mecanismos de acceso al medio compartido, de control de flujo, retransmisiones de tramas, etc.

Red Se encarga de encaminar los mensajes a su destino, con posibles mejoras como el control de congestión. Puede ser orientado a conexión o no.

Transporte Primera que conecta directamente a los equipos finales. Divide la información si es necesario antes de pasarla a la capa de red, y se encarga de que llegue correctamente.

Sesión Permite establecer sesiones mediante funciones como la sincronización en el envío de la información.

Presentación Incluye otras funciones que las aplicaciones requieren con frecuencia.

Aplicación Transferencia de ficheros, correo electrónico, directorios, etc.

Este modelo resultó de una labor muy rigurosa de estudio, separación de funcionalidad, definición clara de servicio, interfaz y protocolo, etc., pero para cuando estuvo listo, **TCP/IP** ya se había establecido. Este modelo no dice gran cosa de los niveles físico y de enlace. La capa de red es siempre **IP** (*Internet Protocol*), no orientada a conexión, y la de transporte puede ser **TCP** (*Transmission Control Protocol*), confiable y orientada a conexión, o **UDP** (*User Datagram Protocol*), no confiable ni orientada a conexión. No tiene capas de sesión ni de presentación, que en el modelo OSI se usaron poco, y la de aplicación incluye protocolos como los de terminal virtual (Telnet), transferencia de ficheros (FTP) o correo electrónico (SMTP).

Internet se compone de miles de redes organizadas de forma jerárquica pero muchas veces solapadas. Generalmente, los *hosts* se agrupan en **redes de área local** (LAN), que se conectan a la red de un proveedor de servicios de Internet (**ISP**, *Internet Service Provider*) mediante la conexión a un punto de presencia (**PoP**, *Point of Presence*) a través de un equipo proporcionado por la ISP (**CPE**, *Customer Premises Equipment*).

Por cierto, el correo postal funciona por conmutación de mensajes, no tiene garantía de tiempo de entrega o de entrega en orden, y los mensajes pueden perderse.[1]

DRAFT

[1] No sé a qué viene esto, pero estaba en las diapositivas.

Capítulo 2

Algoritmos y protocolos de encaminamiento

Los dispositivos de Internet se comunican mediante paquetes **IP**, que en versión 4 (**IPv4**) tienen formato:

0 – 3	4 – 7	8 – 15	16 – 18	19 – 31
Versión	L. cabec.	Tipo de servicio	Longitud total	
Identificación			Flags	Desplazamiento de fragmento
Tiempo de vida		Protocolo transporte	Suma de verificación de cabecera	
Fuente				
Destino				
Opciones				Padding

DRAFT

Las direcciones IPv4 tienen 32 bits que se representan en grupos de 8 bits en decimal separados por puntos. Para no tener que guardar el puerto de salida correspondiente a cada dirección IP individualmente, estas se organizan en redes: los primeros bits de la dirección representan la red y el resto representan una interfaz de dispositivo dentro de la red.

Dos equipos pueden comunicarse directamente si están en la misma red, y de lo contrario hace falta al menos un *router* intermedio que conecte las redes. Se usa el protocolo **ARP** para averiguar la dirección de enlace correspondiente a la dirección IP de otro equipo en la misma red, que será el equipo al que se quiere enviar o el primer *router* intermedio.

Una **máscara de red** indica cuántos bits corresponden a cada parte, y se expresa como una dirección en que los bits de red están a 1 y los de interfaz a 0, o de la forma $/n$ donde n es el número de bits de la parte de red. Dentro de una red, la dirección con todos los bits de interfaz a 0 es la **dirección de red**, que identifica a la red, y la que tiene todos los bits a 1 es la **dirección de *broadcast***, para enviar a todos los dispositivos de la red.

Inicialmente, las redes se dividen en clases:

Clase	Bits iniciales	Máscara	Clase	Bits iniciales	Uso
A	0	/8	D	1110	Multidifusión
B	10	/16	E	1111	Reservado
C	110	/24			

Como esto es poco flexible y desperdicia muchas direcciones, se crea el ***subnetting***, permitiendo dividir una red en varias subredes de igual tamaño usando una nueva máscara, y **VLSM** (*Variable Length Subnet Masks*), que permite que las subredes sean de tamaño variable indicando la máscara de red de cada una. También se crea el ***supernetting***, permitiendo fusionar varias redes existentes que se enrutan por el mismo sitio, mediante **CIDR** (*Classless Inter Domain Routing*), eliminando las diferencias entre las clases A, B y C indicando la máscara para todas las redes y subredes.

El **encaminamiento** es el proceso por el que los dispositivos de red hacen que los datagramas circulen por la red hasta alcanzar su destino. Es **fijado en el origen** si el encaminador origen calcula el camino a seguir y lo indica en el datagrama, o **salto a salto** si cada encaminador intermedio decide a qué vecino enviar.

Los *routers* y *hosts* encaminan salto a salto con **tablas de rutas**, cuyas entradas contienen el identificador de una red (dirección y máscara) y la interfaz o dirección IP por la que alcanzarla. Para ello primero consultan las redes por las que el destino es alcanzable directamente, y si no las hay, consultan el resto de redes que contengan el destino, dando error si tampoco las hay.

Si hay varias rutas de igual tipo para el destino se toma la más específica, la que contenga más bits de red (***Longest Prefix Match***). Una red con máscara /32 se refiere realmente un host (*host-specific route*), y suele haber una **ruta por defecto** usando 0.0.0.0/0.

DRAFT

2.1. Algoritmos de encaminamiento

Son formas de establecer las entradas de la tabla de rutas según criterios como mínimo coste económico o retardo o máximo ancho de banda.

Se requiere **corrección**, obtención de rutas que efectivamente llegan al destino. También son importantes la **robustez**, capacidad de converger a una solución tras posibles fallos de hardware y software en la red, y **simplicidad**, facilidad para entender, implementar y depurar el algoritmo. También son deseables la **optimalidad**, obtención del mejor camino al destino; **estabilidad**, convergencia rápida a una solución tras cambios de estado en la red, y **equidad**, capacidad de ofrecer el mismo servicio a todos los usuarios.

Un algoritmo de encaminamiento es **estático** si se usa información previamente disponible para fijar la tabla de antemano; **cuasi-estático** si se usa información previa pero, cuando una ruta falla, se recalcula o se usa una alternativa, o **adaptativo** o **dinámico** si tiene en cuenta los posibles cambios en la red y recalcula un nuevo camino si es necesario.

A su vez, un algoritmo dinámico es **centralizado** si se envía todo el estado a un solo nodo encargado de calcular las rutas; **aislado** si cada nodo usa sólo información local, o **distribuido** si los nodos intercambian información para el cálculo de los caminos. Estos últimos son los más usados, superando a los centralizados en escalabilidad, tolerancia a fallos (no tienen un punto único de fallo) y reducción de la sobrecarga de control.

2.1.1. Encaminamiento por inundación

Cada dispositivo reenvía por todas las interfaces salvo por la que recibió el mensaje, que acaba llegando a toda la red y por tanto también al destino. El algoritmo es sencillo y llega al destino por la ruta más corta (entre otras), pero requiere identificar los mensajes para evitar

ciclos, y es ineficiente porque si E es el total de enlaces en la red, el mensaje se envía por red un total de entre E y $2E$ veces, inclusive.

Para inundar de forma más eficiente podemos tratar la red como un grafo conexo y crear un **árbol de expansión** (*spanning tree*), un árbol y subgrafo que incluye a todos los nodos, partiendo del nodo emisor. Si hay V nodos, el total de envíos es $V - 1$, y además existe un camino entre cada par de nodos en el árbol.

2.1.2. Encaminamiento por el camino más corto

Si la red es un grafo conexo (V, E, w), siendo $w : E \to \mathbb{R}^{\geq 0}$ la función de coste de los enlaces (retardo, coste económico, etc.), llamamos **coste** de un camino $(v_0, \ldots, v_n)$ a $\sum_{k=1}^{n} w(v_{k-1}, v_k)$. El **principio de optimalidad** nos dice que si $(v_0, \ldots, v_n)$ es el camino más corto del nodo v_0 a v_n, para $0 \leq i < j \leq n$, $(v_i, \ldots, v_j)$ es el camino más corto de v_i a v_j. Así, los enlaces que pasan por caminos mínimos de algún nodo de la red hacia un destino dado forman un árbol de expansión cuya raíz es el destino.

Podemos hallar este árbol con el algoritmo de Dijkstra o el de Bellman-Ford, pero el resultado no es óptimo porque, aunque haya varios caminos entre dos nodos, el tráfico solo viajaría por el más corto. Para conseguir máximo rendimiento haría falta *optimal routing*, separando el tráfico en múltiples caminos en puntos estratégicos según la teoría de *optimal multi-commodity flows*.

DRAFT

2.1.3. Algoritmo de vector de distancias

Periódicamente, cada nodo construye un vector con lo que le costaría llegar al resto de nodos conocidos y lo envía a sus vecinos. Cuando recibe esta información de sus vecinos, para cada dirección, suma el coste recibido y el coste del enlace para obtener lo que le costaría a él, y guarda el mínimo de estos costes junto con la interfaz asociada, con la excepción de que el coste para enviarse a sí mismo es 0.

Una vez el algoritmo converge, si falla un enlace, un nodo deja de recibir información de su vecino, pero sí que recibe el coste calculado por otros vecinos sobre el vecino, aunque la ruta de estos vecinos pase por el propio nodo, causando un ciclo. Si el vecino queda inalcanzable, el resto no lo sabrá, sino que el coste que calculen seguirá aumentando en cada ronda. Este es el **problema de la cuenta a infinito**, y existen varias soluciones:

- La trivial: si el coste a un nodo supera un cierto valor pequeño, este se considera inalcanzable.
- ***Split horizon***: No se anuncia el coste a un destino al router que representa el mejor camino al destino.
- *Split horizon* con ***poison reverse***: En vez de no anunciar el coste, se anuncia un coste infinito.

2.1.4. Algoritmo de estado de enlace

Periódicamente, cada router construye un *Link State Packet* (**LSP**) con su identificador, un número de secuencia, la lista de sus vecinos y las distancias a cada uno, e inunda la red con

el mensaje. Cuando recibe esta información del resto de routers, calcula el camino más corto al resto, normalmente generando un árbol de expansión mínimo.

Para evitar que circulen mensajes antiguos, cuando llega un LSP con número de secuencia menor o igual al mayor de los vistos hasta el momento para el mismo origen, este se descarta.

Con este algoritmo, los mensajes son más pequeños que en vector de distancias, pero requieren más envíos. La necesidad de memoria es mayor debido a que un LSP ocupa más que un elemento del vector distancia, pero a cambio el algoritmo es más fiable y eficiente, usándose en protocolos como OSPF o IS-IS.

2.1.5. Dominios de encaminamiento

Si la red es muy grande, tanto el algoritmo de vector de distancias como el de estado de enlace producirían una sobrecarga de control, por lo que en Internet, en vez de calcular las rutas a cada dirección de destino, esto se hace para cada red destino, asumiendo que a través de cualquiera de los routers de esa se se puede alcanzar a todos los hosts.

Las redes se agrupan en **sistemas autónomos** (**AS**), conjuntos de redes administradas por una entidad común que comparten una estrategia de encaminamiento, distinguiendo así entre **encaminamiento interdominio** entre sistemas autónomos distintos mediante un *External Gateway Protocol* (**EGP**) y **encaminamiento intradominio**, dentro de un mismo sistema autónomo mediante un *Internal Gateway Protocol* (**IGP**).

Esta división mejora la escalabilidad, al evitar que todos los routers de Internet tengan que intercambiar mensajes entre sí, y permite usar el protocolo de enrutamiento interno que se desee mejorando así la compatibilidad y extensibilidad.

DRAFT

2.2. RIP

El *Routing Information Protocol* es un protocolo de vector de distancias con primera versión definida en el RFC 1058 que por defecto asigna un coste 1 a cada enlace. Formato de los mensajes:

0 – 7	8 – 15	16 – 31
Comando	Versión	Todo 0s
Familia		Todo 0s
Dirección de red		
v2: Máscara de red		
v2: Dirección del siguiente salto		
Distancia (16 para red inalcanzable)		

Los primeros 32 bits del mensaje son la cabecera, y el resto puede repetirse hasta 25 veces. En la versión 1, los campos de máscara de red y dirección del siguiente salto están a 0. RIP distingue entre participantes activos, que envían y reciben vectores de distancias, y pasivos, que solo reciben. Hay dos tipos de mensaje:

- ***RIP request***: Lo envían los nodos que acaban de conectarse a la red o llevan un tiempo desconectados.

- ***RIP response***: Lo envían los participantes activos como respuesta a un *RIP request*, cada 30 segundos o cuando hay un cambio en su vector de distancias.

Se usa **histéresis**, consistente en solo actualizar la ruta anterior y enviar el mensaje correspondiente si el coste de la nueva ruta es estrictamente menor que el actual, y ***triggered updates***, consistente en enviar un mensaje cuando haya cambiado el coste de alguna entrada sin tener que esperar al siguiente envío periódico. También soporta *split horizon* y *poison reverse*. Para mejorar la convergencia en redes grandes se puede activar el ***hold down timer***, consistente en ignorar todo mensaje sobre una red inalcanzable durante 60 segundos.

Es un protocolo sencillo y disponible en casi cualquier router, pero tiene convergencia lenta, métrica poco dinámica y puede crear bucles.

2.3. OSPF

Es un protocolo basado en estado de enlace, cuya versión 2 se define en el RFC 2328. Reacciona de forma rápida a cambios topológicos, soporta CIDR, y puede encaminar con múltiples métricas, balancear carga entre distintos caminos, organizar un sistema autónomo en áreas de menor tamaño, integrarse con redes externas y mejorar su eficiencia en las internas eligiendo routers designados.

Un AS se divide en una o más **áreas**, de las cuales una es el **área *backbone***, con identificador 0, y el resto conectan con ella. Se clasifican los routers en:

- **Internos**: Conectan a una sola área y mantienen su topología.
- **Frontera de área** (**ABR**): Conectan varias áreas y mantienen la topología de cada una.
- ***Backbone***: Conectan con alguna red del área 0.
- **Frontera de AS** (**ASBR**): Conecta con redes externas al AS.

DRAFT

Una ruta es **intra-área** si el origen y el destino están en la misma área, **inter-área** si están en áreas distintas de un mismo AS o **inter-AS** si el origen y el destino están en ASs distintos.

Si hay varios routers OSPF en el mismo segmento de red, para reducir la sobrecarga de control, se eligen un **router designado** (**DR**) y un **backup del router designado** (**BDR**), con lo que el resto de routers de este segmento mantienen adyacencias con estos dos que a su vez mantienen una adyacencia entre ellos.

Para elegir un router designado, si ya existe uno se mantiene; si ya existe un BDR pero no un DR, este pasa a ser DR, y de lo contrario se elige como DR el router con mayor prioridad, establecida por el administrador, o en caso de empate de prioridad, el que tenga un mayor ID de router, y se establece el BDR como el siguiente router según estos criterios.

Los mensajes tienen cabecera con el formato:

0 – 7	8 – 15	16 – 32
Versión	Tipo	Longitud de paquete
ID del router		
ID del área		
Suma de verificación		Tipo de autenticación
Autenticación		
⋮		

El tipo puede ser:

1. ***Hello***: Cada router envía periódicamente (por defecto cada 10 segundos) un mensaje de este tipo por todas sus interfaces a *AllOSPFRouters* (224.0.0.5) con TTL 1, listando sus vecinos conocidos, y considera a un vecino inalcanzable si no recibe mensajes de *Hello* de él en un tiempo (por defecto 40 segundos).

2. ***Database description***: Cuando dos routers OSPF se ven por primera vez, lo que detectan con mensajes *Hello*, se envían mensajes de este tipo en unicast con la lista de LSAs en sus bases de datos (no su contenido).

3. ***Link State Request***: Cuando un router recibe un *Database Description* con algún LSA que no tiene o de la que solo tiene una versión anterior, envía este mensaje en unicast para solicitar los LSAs que necesita.

4. ***Link State Update***: Contienen uno o más **LSAs** (*Link State Advertisements*), la información sobre el estado de los enlaces de un router. Se envían a *AllDRouters* (224.0.0.6) en respuesta a un *Link State Request*, tras detectar un cambio en los enlaces con adyacencias o para inundar LSAs recibidos en un *Link State Update*.

5. ***Link State Ack***: Confirma la recepción de un *Link State Update*.

Un mismo *Link State Update* se envía a 224.0.0.5 con TTL 1, y puede contener LSAs de distintos tipos:

1. ***Router LSA***: Contiene el ID del router que lo generó, sus enlaces y un bit que indica si el router es un ASBR, y se inunda dentro de un área.

2. ***Network LSA***: Lo envían los DRs, contienen las direcciones IP del resto de routers OSPF del medio compartido y se inundan dentro del área. Circulan sólo si hay segmentos de red de medio compartido con varios routers OSPF.

3. ***Network summary LSA***: Lo genera un ABR para cada red dentro de un área y lo envía a sus vecinos de otras áreas. Es un LSA resumen, por lo que no indica la topología del área sino solo el coste desde el ABR y puede fusionar redes con *supernetting*.

4. ***ASBR summary LSA***: LSA resumen que genera un ABR que conecta con un área en la que hay un ASBR, lo que sabe al ver el bit ASBR activado en un LSA de tipo 1 o 2, y que propaga al área 0 para que se inunde en el resto de áreas. Contiene los costes de llegar a los ASBRs desde el ABR.

5. ***External LSA***: LSA resumen que generan los ASBRs, incluyen información de redes fuera del AS y se inundan por todo el AS.

7. ***NSSA LSA***: Con igual función y contenido que los de tipo 5 pero en áreas NSSA, donde los LSA de tipo 5 no están permitidos.

Las áreas NSSA son un tipo de áreas especiales:

- ***Stub area***: No hay ningún ASBR dentro del área, y los ABR filtran los LSAs de tipo 4 o 5 enviando en su lugar uno de tipo 3 con una ruta por defecto.

DRAFT

- ***Totally Stub area***: No hay ningún ASBR en el área y sólo hay un ABR, que filtra los LSAs de tipos 3–5 enviando en su lugar uno de tipo 3 con una ruta por defecto.

- ***Not-so-stubby area*** (**NSSA**): Hay ASBRs, pero generan LSAs de tipo 7 en vez de 5, y los ABRs filtran los LSAs de tipo 5 enviando en su lugar uno de tipo 3 con una ruta por defecto. Así, si una red es alcanzable tanto por un ASBR de otra área como por uno interno, se elegirá siempre el interno.

- ***Totally NSSA***: Como NSSA, pero los ABRs también filtran los LSAs de tipo 3.

Originalmente, OSPF calculaba 5 costes distintos según el campo de tipo de servicio de la cabecera IP:

`0x00` Número de saltos.

`0x02` Coste económico fijado por el administrador.

`0x04` Fiabilidad según el histórico reciente de tasa de pérdida.

`0x08` Velocidad de transmisión: duración de un bit en unidades de 10 ns, o 1 si el bit dura menos que esto.

`0x10` Tiempo de transmisión más tiempo de encolado.

Cada router calculaba 5 árboles de expansión. Actualmente esto ya no se hace, y se usa siempre la velocidad de transmisión. Para Ethernet de 10 Mbps y serie de 1544 kbps o 128 kbps, había diferencia, pero a partir de Fast Ethernet, el coste siempre es 1 y la métrica es esencialmente el número de enlaces.

Si el coste empleado por el protocolo de un AS externo usa unidades distintas a las de dentro de nuestro AS, los ASBR hacen una **redistribución**, que puede ser de dos tipos:

1. El coste que anuncia el ASBR se va actualizando conforme se propaga la ruta.

2. El coste se mantiene constante.

DRAFT

2.4. BGP

El *Border Gateway Protocol* es un protocolo de encaminamiento interdominio que tiene como objetivo encontrar un camino entre origen y destino que cumpla las políticas establecidas por cada AS intermedio, pues los AS pueden ser competidores, y que no tenga ciclos. El primer protocolo de este tipo en Internet fue **EGP**, puesto en marcha en 1984. En 1989 se creó la primera versión de BGP. La versión 4 (BGP4) se especifica en el RFC 1771. En 1998 se creó BGP4+, descrita en el RFC 2283, con extensiones para IPv6, multicast y soporte multiprotocolo.

Cada AS tiene un **ASN** (*Autonomous System Number*): Genuity (actualmente BBN) tiene 1; MIT, 2; Harvard, 11; RedIRIS, 766; AT&T, 7018, 6341, etc. Llamamos **tráfico local** de un AS al que se origina o termina en dicho AS, y **tráfico de tránsito** al que no es local. Respecto a sus vecinos, un AS puede ser:

Stub Solo se conecta a un AS externo, por lo que solo encamina tráfico local.

Multihommed Conecta con dos o más ASs pero no cursa tráfico de tránsito.

Transit Cursa tanto tráfico local como de tránsito. Un AS puede ser *transit* para algunos y *multihommed* para otros.

BGP es un protocolo de **vector de caminos**, similar a uno de vector de distancia pero, en vez de comunicar costes, comunica caminos completos. Como no tiene sentido calcular un camino globalmente óptimo porque no se conoce la topología interna de los ASs y el camino con menos ASs no siempre es el más corto, lo que realmente se anuncia es alcanzabilidad.

Un anuncio indica al AS que lo recibe que puede enviar tráfico por la ruta indicada. Un router BGP solo puede anunciar una ruta a cada red, con lo que si recibe varias rutas al mismo destino, se queda con una (o ninguna) y la anuncia a cero o más vecinos.

Una sesión BGP empieza estableciendo una sesión con el puerto TCP 179. Después los routers se intercambian las rutas activas y, a continuación, las actualizaciones incrementales. Hay 4 tipos de mensajes:

Open Establece la sesión.

Keep Alive Se envía cada cierto tiempo para informar al vecino de que la sesión sigue activa.

Notification Cierra la sesión.

Update Contiene los anuncios de nuevas rutas o rutas que deben ser anuladas.

DRAFT

Los anuncios de rutas constan de una serie de prefijos de red (dirección IP y máscara de red) junto a una serie de atributos (ID numérico y valor), como pueden ser:

ID	Código	Referencia	ID	Código	Referencia
1	`ORIGIN`	RFC 1771	9	`ORIGINATOR_ID`	RFC 2796
2	`AS_PATH`		10	`CLUSTER_LIST`	
3	`NEXT_HOP`		11	`DPA`	Chen
4	`MULTI_EXIT_DISC`		12	`ADVERTISER`	RFC 1863
5	`LOCAL_PREF`		13	`RCID_PATH` o `CLUSTER_ID`	
6	`ATOMIC_AGGREGATE`		14	`MP_REACH_NLRI`	RFC 2283
7	`AGGREGATOR`		15	`MP_UNREACH_NLRI`	
8	`COMMUNITY`	RFC 1997	16	`EXTENDED_COMMUNITIES`	Rosen

Los más importantes son 2–5 y 8. `AS_PATH` contiene el vector de caminos, y `NEXT_HOP` contiene la IP del router frontera que envió el anuncio para que este sepa qué poner en su tabla de rutas para los paquetes de la red indicada.

Los routers frontera de un mismo AS deben tener una visión coordinada para que, por ejemplo, una ruta a una red alcanzable a través de uno de los routers frontera pueda ser anunciado por otro. Para esto, todos los routers frontera de un mismo AS mantienen conexiones BGP con el resto para intercambiar las redes que encuentran. Hablamos de **iBGP** (*internal BGP*) cuando la conexión BGP es entre routers del mismo AS y de **eBGP** (*external BGP*) cuando es entre routers de ASs distintos.

Capítulo 3

IP Multicast

Permite enviar un datagrama IP a muchos receptores enviando a una dirección IP especial en el rango 224.0.0.0/4 a la que los receptores se suscriben. Esto reduce el consumo de ancho de banda y la carga tanto en servidores, al no tener estos que hacer réplicas del contenido, como en la red, aunque requiere usar UDP dado que TCP requiere asentimientos y esto puede saturar la red si hay muchos receptores.

Direcciones *multicast* reservadas:

- ***Link-local***: 224.0.0.0/24, para protocolos que se ejecutan en un segmento local de red. Los routers nunca los reenvían.
- 224.0.1.0/24, reservadas para protocolos. Por ejemplo, la dirección .1 se asigna a **NTP** (*Network Time Protocol*).
- ***Administratively Scoped Multicast Addresses***: 239.0.0.0/8, para uso privado dentro de una organización.

DRAFT

Cualquier otra dirección puede ser usada libremente: un equipo puede enviar a una dirección *multicast* sin requisito previo y otros pueden suscribirse. El control de ámbito de paquetes *multicast* privados se puede hacer mediante:

- ***TTL Scoping***: Configurar los routers para no dejar pasar por ciertas interfaces paquetes con TTL inferior a uno dado, y configurar los hosts para enviar los paquetes privados con menor TTL.
- ***Administrative Scoping***: Restringir que ciertos rangos de direcciones pasen por ciertas interfaces. Menos usado al ser menos intuitivo.

En Ethernet se preveía la posibilidad de tramas *multicast* o *broadcast*, que se diferenciaban de las *unicast* porque el bit menos significativo del primer octeto de la dirección MAC era 1. Sin embargo, cuando Steve Deering hacía su tesis sobre MAC *multicast*, Jon Postel solo pudo comprar un prefijo MAC de 24 bits (**OUI**, *Organizational Unique Identifier*) y ceder la mita a Steve, por lo que ahora las tramas *multicast* tienen una MAC destino de la forma `01:00:5e:xx:xx:xx`, con la primera `x` entre 0 y 7.

La conversión de IP *multicast* a MAC *multicast* se hace añadiendo a este prefijo los 23 bits menos significativos de la dirección IP, con la que $2^5 = 32$ grupos *multicast* se mapean a la

misma MAC. Si dos equipos en una misma subred se suscriben a grupos distintos con igual MAC, cada uno puede recibir tráfico del otro grupo, que se descarta a nivel de red.

En 1992 se creó *Multicast Backbone* o MBone, una red virtual experimental sobre Internet para experimentar con el despliegue de IP *multicast*, que por entonces los routers no sabían encaminar. En las redes de MBone, los MRouters encapsulaban datagramas *multicast* en datagramas *unicast* dirigidos a otros MRouters, vecinos a nivel de IP *multicast* pero no a nivel *unicast*. Con el tiempo esto dejó de ser necesario.

3.1. IGMP

Es el protocolo que permite a un router *multicast* conocer en qué redes hay receptores unidos a ciertos grupos multicast. Además, algunos *switches* implementan ***IGMP Snooping***, consistente en detectar los mensajes IGMP para enviar los paquetes *multicast* sólo por donde hay receptores interesados.

3.1.1. IGMPv1

Descrito en el RFC 1112, funciona sobre IP con tipo de protocolo 0x02 y TTL 1. Formato:

0 – 3	4 – 7	8 – 15	16 – 31
1	Tipo	Reservado	Checksum
Dirección *multicast*			

El tipo puede ser:

1. ***Query*** o ***general query***, enviado por los routers de forma periódica, por defecto cada 60 segundos, por sus interfaces a 224.0.0.1 (AllHosts), para preguntar a los hosts a qué grupos desean unirse.

2. ***Report***, enviado por un host a un grupo *multicast* con dirección *multicast* igual a la de dicho grupo para indicar pertenencia al grupo. Los hosts lo envían al unirse un grupo y, al recibir una *query* de un router, para cada grupo al que pertenecen, esperan un tiempo aleatorio entre 0 y un **MRT** (*Maximum Response Time*) de 10 segundos y, si ningún otro host ha enviado un *report* para ese grupo en ese tiempo, lo envían.

Los hosts abandonan el grupo sin avisar, con lo que si para un grupo el router no recibe respuesta tras enviar un número de *queries*, digamos 3, justo antes del siguiente *query* deja de enviar tráfico multicast de dicho grupo.

3.1.2. IGMPv2

Definida en el RFC 2236, es la versión más usada del estándar. Se encapsula sobre IP con el mismo tipo de protocolo que IGMPv1. Formato:

0 – 7	8 – 15	16 – 31
Tipo	MRT	Checksum
Dirección *multicast*		

El campo MRT permite al router indicar el ***Query Interval Response Time***, el tiempo máximo que espera el router respuestas a *queries*, en décimas de segundo con valor por defecto de 10 segundos. El tipo puede ser:

DRAFT

`0x11` ***Membership Query***, como una *query* de IGMPv1, pero si la dirección *multicast* no es 0, no es una *general query* sino una ***Group-Specific Query***, que pregunta por un grupo concreto.

También hay un mecanismo de ***Querier Election*** para elegir qué *router* es el ***Querier***, envía las *queries*: inicialmente todos envían, y si ven a otro con una IP menor enviando, dejan de enviar. El ***General Query Interval***, el tiempo entre *queries* consecutivas, aumenta a 125 segundos, y si los *routers* que no envían *queries* (***non-Querier***) dejan de escuchar al *Querier* durante dos veces este intervalo, se reinicia la elección.

`0x12` ***Membership Report v1***.

`0x16` ***Membership Report v2***.

`0x17` ***Leave Group***. Lo envía un host a 224.0.0.2 (AllMcastRouters) para indicar que abandona un grupo. Entonces el *Querier* envía un *Group-Specific Query* a dicho grupo para saber si hay más hosts suscritos, y si tras dos intentos no responde nadie, se da el grupo por vacío.

3.1.3. IGMPv3

Definido en el RFC 3376, permite que un host se pueda unir a un grupo solo para algunas fuentes (*source filtering*), útil en *Source Specific Multicast* (**SSM**), comunicaciones multicast con solo una fuente y muchos receptores.

DRAFT

3.2. Encaminamiento

Destacamos varios tipos de **árbol *multicast***:

- ***Steiner tree***: La suma de los costes de todos los enlaces es mínima, pero su cálculo es NP-completo.
- ***Shortest Path Tree*** (**SPT**): Unión de los caminos más cortos entre la fuente y cada receptor. Si S es el número de fuentes activas por grupo y G el número de grupos activos pasando por un cierto router, como cada fuente y grupo requieren un árbol distinto, cada router deberá almacenar unas SG entradas.
- ***Shared Tree*** (**ST**): Se usa el mismo árbol para varias fuentes, formado por los caminos más cortos desde un *router* que hace de raíz del árbol a todas las fuentes y receptores. Como se usa el mismo árbol para varias fuentes, los routers intermedios almacenan G entradas.

El ***Reverse Path Forwarding*** es una técnica usada para evitar ciclos consistente en comprobar si el mensaje lleva de donde debería (***RPF Check***), que es la interfaz que él usaría para llegar a la fuente en caso de SPT o a la raíz en ST, y no reenviar en caso contrario.

El reenvío se hace a través de la ***Multicast Forwarding Cache***, una lista de entradas de la forma (S, G, I) que representan cada una la visión local de un árbol *multicast*, donde S es la IP de una fuente, G la de un grupo *multicast* e I la ***Outgoing Interface List*** (**OIL**), el

conjunto de interfaces por las que reenviar el datagrama. En caso de un árbol ST, la fuente es *.

Tipos de protocolos de encaminamiento *multicast*:

- ***Dense mode***, como DVMRP y PIM-DM. Asumen un gran número de receptores para cada grupo. Funcionan por **inundación y poda**: periódicamente, el tráfico de una fuente se inunda por toda la red evitando ciclos, y las ramas del árbol sin receptores interesados se podan. Si entonces aparece un nuevo receptor, se usa un mensaje de **injerto** (*graft*) para volver a indicar interés en el tráfico *multicast* sin tener que esperar a la siguiente fase de inundación. Estos protocolos suelen construir SPTs, pero tienen como principal problema la escalabilidad. Son poco usados hoy en día debido al coste de la inundación y poda.

- ***Sparse mode***, como PIM-SM y CBT: Asumen pocos receptores en cada grupo. Los routers conectados a redes con receptores para un cierto grupo solicitan la creación de un camino (***explicit join***) propagando un mensaje de unión al grupo, que hace que los routers intermedios hasta la raíz del árbol añadan la interfaz correspondiente a la OIL. Son más escalables, pero los árboles no son siempre óptimos.

- **Híbridos**, como Multicast OSPF: Inundan la red periódicamente con información de encaminamiento, pero el tráfico *multicast* solo se envía por las ramas correspondientes.

3.3. DVMRP

Distance Vector Multicast Routing Protocol, definido en el RFC 1075, es un protocolo *dense mode* en que periódicamente, por defecto cada 60 segundos, se olvidan las ramas podadas y se vuelve a inundar la red con tráfico *multicast*.

Antes de reenviar tráfico *multicast*, los routers construyen una tabla de rutas *multicast* intercambiando vectores de distancia de forma similar a RIP, con 32 para indicar infinito y *Poisson Reverse*, salvo que cuando un router recibe un anuncio de coste infinito a cierta fuente, deduce que él es el padre del vecino en el árbol SPT para la fuente dada y por tanto añade al vecino en las OILs correspondientes.

Hay 5 mensajes, que viajan en paquetes IGMP con tipo 0x13:

Probe Lo envían periódicamente los routers DVMRP a 222.0.0.4 (AllDVMRPRouters) por todas sus interfaces para descubrir a sus vecinos. Incluyen una lista de vecinos conocidos.

Report Comunica un vector de distancias.

Prune Solicita salir del árbol de cierto grupo y fuente por no tener receptores interesados ni hijos en el árbol.

Graft Solicita ser añadido al árbol de cierto grupo y fuente, al volver a tener receptores interesados o hijos en el árbol.

Graft Ack Confirma la recepción de un mensaje *Graft*.

Aunque es raro que a un router le llegue un datagrama *multicast* por una interfaz distinta a la que le une con su padre, sigue siendo necesario aplicar el ***RPF Check*** porque, ante un cambio topológico, la convergencia del algoritmo de vector de distancias es lenta.

DRAFT

3.4. PIM-SM

Protocol-Independent Multicast, Sparse Mode, definido en el RFC 4601, es un protocolo *sparse mode* en que a cada grupo *multicast* se le asocia un ***Rendezvous Point*** que hace de raíz de un árbol ST, aunque también permite crear árboles SPT. El RP puede configurarlo manualmente un administrador de red o aprenderse mediante un protocolo como *PIMv2 Candidate RP Advertisements* o el protocolo privativo de Cisco *Auto-RP*.

Mensajes:

Hello Enviado periódicamente, por defecto cada 30 segundos, a 224.0.0.14 con TTL 1, por todos los routers PIM, con la dirección IP del router que lo envía, para que se pueda elegir entre todos los routers de cada subred un **router designado** (**DR**), que será el que tenga la IP mayor.

Join Solicita unirse a un árbol *multicast*. Puede ser $(*, G)$ para un ST o (S, G) para un SPT. El administrador de red puede configurar un ***SPT Threshold***, una tasa de tráfico a partir de la cual se pasa del ST a un SPT, de 0 (cambiar en cuando se reciba tráfico *multicast* de una fuente) a infinito (usar siempre el ST). Cuando se supera, el router envía un mensaje *Join* (S, G) por la interfaz que le ofrece un camino a S, y los routers intermedios van añadiendo la interfaz correspondiente a la OIL hasta llegar al DR de S o a un router intermedio que forme parte del SPT. Este proceso hace que se almacene más estado en las tablas de encaminamiento pero permite que el tráfico *multicast* llegue por los caminos más cortos.

Prune Como *Join* pero para abandonar un grupo. Cuando no hay receptores interesados en un grupo y la OIL queda vacía, se envía un *Prune* $(*, G)$ al padre, que deberá eliminar la interfaz correspondiente de su OIL (***RPT prunning***). Cuando esto ocurre para una cierta fuente y estamos en el SPT correspondiente, se envía un *Prune* (S, G) al padre en el SPT (***SPT prunning***). Cuando un router pasa de un ST a un SPT para una fuente, envía un mensaje *Prune* (S, G) en dirección al RP y añade un bit RP para que se sepa que no se trata de *SPT prunning*. El mensaje se propaga pudiendo llegar al RP.

Graft Similar a DVMRP, solo usado en PIM-DM.

Graft-Ack Similar a DVMRP, solo usado en PIM-DM.

Assert Usado en redes con varios routers PIM para elegir solo a uno para que se una al árbol.

Register Enviado por un DR que acaba de recibir tráfico *multicast* directamente de una fuente para indicar al RP que la fuente está activa y que este pueda crear la rama del árbol correspondiente enviando un mensaje *Join* (S, G) por la interfaz que le lleva hacia la fuente. El mensaje se envía por *unicast* del DR al RP, y encapsula el paquete *multicast* original.

Register Stop Enviado por el RP al router de la fuente para decirle que ya puede dejar de duplicar el tráfico *multicast* encapsulándolo en mensajes *unicast Register*.

Para evitar que los routers PIM-SM puedan quedar con información vieja o incorrecta, a las entradas de la *Multicast Forwarding Cache* se les da un tiempo de vida de 3 minutos desde su última actualización, y para evitar que las entradas expiren en un nodo padre, un router se encarga de reenviar los mensajes *Join* y *Prune* periódicamente.

DRAFT